Decision Trees and Pegasos SVMs from Scratch: Price and Price-Tier Prediction on bina.az Listings

[Surname, Name] [Surname, Name] [Surname, Name] [Surname, Name]

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We turn 100,775 raw bina.az sale listings into a leakage-free dataset of 63,525 flats, houses and plots and study two tasks: regression of the sale price (Task A) and classification of a derived premium/standard price tier (Task B). The decision tree (Gini, entropy, MSE) and the soft-margin SVM (Pegasos sub-gradient solver, linear and random-Fourier-feature RBF) are written from scratch in NumPy; scikit-learn is used only as a baseline. On the held-out test set our tree reaches $R^2 = 0.550$ (RMSE 298,954 AZN) on Task A, matching scikit-learn (0.550), and F1 $= 0.910$ on Task B (scikit-learn 0.910). Our linear Pegasos SVM reaches F1 $= 0.850$ and ROC-AUC 0.925, indistinguishable from `SGDClassifier` (0.849, 0.925), while the random-feature RBF SVM adds only $+0.003$ AUC. The tree beats the linear SVM in accuracy because location (latitude/longitude) acts non-linearly, but the SVM ranks listings as well as the tree. Errors concentrate near the tier threshold, on land plots and commercial objects, and in sparse districts. Code: `github.com/your-team/your-repo` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, real-estate price prediction.

I. Introduction

Property prices are heavy-tailed and depend on size, type and location. We use the bina.az sale dump to (i) implement a decision tree and an SVM *from first principles*, (ii) clean a genuinely messy scraped dataset end to end, and (iii) benchmark honestly against scikit-learn. Contributions: a tested NumPy tree supporting Gini/entropy/MSE and four stopping rules; a Pegasos SVM with epochs, learning-rate schedules, margin and support-vector reporting and an RBF extension; a leakage-free pipeline; and a one-command reproduction (`python -m src.run_all`).

II. Related Work / Background

A tree greedily picks the split with the largest impurity decrease $\mathrm{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$ with $I \in \{\text{Gini, entropy, variance}\}$. The soft-margin SVM minimises the regularised hinge-loss primal; Pegasos [1] solves it by stochastic sub-gradient descent with step $\eta_t = 1/(\lambda t)$, and random Fourier features [2] approximate the RBF kernel so the same linear solver gives a non-linear classifier.

III. Data and Preprocessing

Cleaning. The dump has 100,775 rows and 51 columns with Azerbaijani names (translated: `Sahe`→area, `Otaq sayi`→rooms, `Torpaq sahesi`→land_area, ...). Each step is logged by `clean()`: (1) only the latest scrape of each listing is kept (same URL scraped repeatedly; $-36{,}321$ rows) because a repost could sit in train and test; (2) leakage and identifier columns are dropped; (3) currency is normalised to AZN (all rows are AZN), numeric strings are parsed, areas in *sot* become m^2, "7 / 9" is split into floor and floor_total; (4) near-duplicates (same price, area, rooms, floor, coordinates, category and district under a different URL) are removed (-723); (5) rows with price outside $[5\,000,\ 10^7]$ AZN, area outside $[10, 10^5]$ m^2 or more than 15 rooms are dropped (-206); (6) coordinates outside Azerbaijan become missing. Result: 63,525 rows.

Leakage. `unit_price` and `total_price` are functions of the target (`total_price` equals `price` in 99.99% of rows) and are removed, as are owner/shop names, address, description, URLs and ids. Median imputation, category levels, standardisation statistics and the tier threshold are computed on the training split only.

Table 1. Dataset summary after cleaning.

Property	Value
Rows (raw → clean)	100,775 → 63,525
Features used	91
Train / Val / Test split	44,467 / 9,529 / 9,529
Premium share train/val/test (%)	49.4 / 49.2 / 49.2
Tier threshold (train median)	210,000 AZN

Tasks and split. Task A predicts `price`; Task B predicts premium ($= 1$ if price $>$ 210,000 AZN, the *training* median). Rows are split 70/15/15, stratified on price quartiles with seed 42 (Table 1). Rare category levels (fewer than 100 listings) are merged into "other"; one-hot encoding gives 91 features.

IV. Method

A. Decision tree (from scratch)

`DecisionTree` mirrors the scikit-learn API (`fit`, `predict`, `predict_proba`). For every feature the values are sorted once and cumulative class counts (or cumulative $\sum y$, $\sum y^2$ for MSE, on centred targets to avoid cancellation) give the impurity of *all* candidate cuts at once; thresholds are midpoints between distinct values. Growth stops at `max_depth`, `min_samples_split`, `min_samples_leaf` or when the weighted gain $\frac{N_t}{N}\Delta I <$ `min_impurity_decrease`; each parameter provably changes the tree (unit test). Leaves predict the majority class (class frequencies for `predict_proba`) or the mean. Impurity-decrease feature importances are normalised to sum to one.

Table 2. Task A: regression of price (test set, mean ± bootstrap std; fit time in s).

Model	RMSE	MAE	R^2
Ours: tree	298,954 ± 14,451	110,300 ± 2,763	0.550 ± 0.037
sklearn: tree	298,973 ± 14,448	110,313 ± 2,763	0.550 ± 0.037
Ours: tree (log y)	314,671 ± 17,102	100,915 ± 3,005	0.501 ± 0.026

5-fold CV: ours $R^2 = 0.563 \pm 0.024$, sklearn 0.563 ± 0.024

B. *Support-vector machine (from scratch)*

`PegasosSVM` minimises

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big) \qquad (1)$$

with the update $\mathbf{w} \leftarrow (1 - \eta_t \lambda)\mathbf{w} + \frac{\eta_t}{k}\sum_{i \in V} y_i \mathbf{x}_i$ over mini-batches of $k = 64$ examples (V = margin violators). The bias is folded in as a constant feature, $\mathbf{w}$ is projected onto $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$, and the second half of the iterates is averaged. Exposed hyper-parameters: λ, number of epochs (or steps) and the learning-rate schedule ($\eta_t = \eta_0/(\lambda t)$, constant, or $\eta_0/\sqrt{t}$). Features are standardised with training statistics. We report $\|\mathbf{w}\|$, the margin $2/\|\mathbf{w}\|$ and the support vectors ($y_i f(\mathbf{x}_i) \le 1$). For non-linearity we chose random Fourier features ($D = 300$) over kernelised Pegasos because training stays $O(\text{steps} \cdot k \cdot D)$ with no $n \times n$ kernel matrix for $\approx 4.4 \cdot 10^4$ training rows.

V. Experimental Setup

Task A: RMSE, MAE, R^2 on price. Task B: precision, recall, F1 (premium is positive), ROC-AUC, PR-AUC and confusion matrices; all metrics are written in NumPy. Hyper-parameters are chosen on the *validation* split only: tree depth over $\{2, \ldots, 18\}$, criterion, then `min_samples_leaf`; SVM $\lambda \in [10^{-6}, 10^{-1}]$, RBF γ and λ. Because validation F1 differences below 0.002 are noise, for the SVM we take the most-regularised λ within 0.002 F1 of the best. Chosen: Task A tree depth 6, leaf 20; Task B tree entropy, depth 18; linear $\lambda = 10^{-3}$; RBF $\lambda = 10^{-5}$, $\gamma = 0.0027$. Baselines use matched settings: `DecisionTree*` with the same parameters, `SGDClassifier(loss=hinge, α = λ)` and `SVC(C=` $1/(\lambda n)$`)` inside `Pipeline(StandardScaler, ...)`; `SVC` is trained on an 8,000-row subsample for tractability. Test-set uncertainty is a bootstrap std (200 resamples of the test set); we also give 5-fold CV mean ± std on train+val. Seed 42 everywhere.

VI. Results

Tables 2 and 3 give the required comparison (test set; bootstrap std).

Tree vs scikit-learn. Metrics agree to the third decimal. On Task A the two trees have the same depth (6) and number of leaves (43) and make identical predictions for 99.0% of test rows; on Task B (depth 18) the leaf counts are 2,436 vs 2,450 and 97.9% of predictions coincide. The differences come from tie-breaking between equally good splits and from float rounding in the cumulative-sum impurity, as expected. Our fit is about 5–25× slower: scikit-learn uses compiled Cython, ours uses vectorised NumPy per node and a Python loop in `predict`.

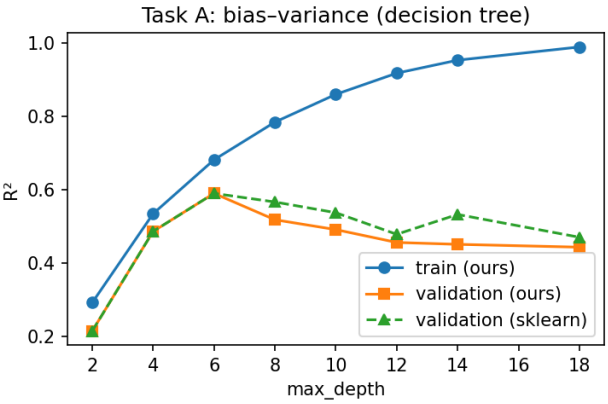


Figure 1. Task A validation curve: depth 6 is optimal; deeper trees over-fit.

Bias–variance (Fig. 1). In Task A training R^2 rises from 0.29 (depth 2) to 0.99 (depth 18) while validation R^2 peaks at 0.59 at depth 6 and falls to 0.44: classic over-fitting. In Task B validation F1 saturates near 0.91 after depth 10 (train 0.99); entropy and Gini differ by up to 0.025 at depths 2–4 (in opposite directions) but are within 0.005 F1 from depth 6 on, with entropy marginally ahead at depths 14–18 (0.915 vs 0.911–0.914); neither criterion is clearly better.

SVM (Fig. 2). Larger λ widens the margin (from 0.01 at $\lambda = 10^{-6}$ to 1.81 at 10^{-1}) and raises the support-vector count (6,708 → 23,966), while validation F1 is flat (0.845) up to $\lambda = 10^{-2}$ and drops to 0.830 at 0.1. The objective J decreases monotonically in expectation and converges (`svm_convergence.png`); more epochs reduce J (2.95 at 0.1 epoch → 0.375 at 30) and lift F1 from 0.822 to 0.845, and the three schedules reach similar J (0.371–0.377) with different margins (0.65 Pegasos, 0.74 $1/\sqrt{t}$, 0.79 constant). The RBF map gives only +0.003 AUC over the linear model: the non-linearity in this problem is mostly in location, which the tree exploits through axis-aligned cuts on latitude/longitude.

VII. Discussion and Limitations

Importances. The tree's Task A importances are dominated by `area` (0.62), the *Obyekt* category (0.15, commercial objects) and latitude (0.08); for Task B `area` (0.45), longitude (0.19) and latitude (0.12) lead (Fig. 3). The largest SVM weights on standardised features are `area` (+1.9) and `rooms` (+1.8); new builds are positive and plots/houses and peripheral districts (Masazır, Hövsan, Mammadli) negative.

Where models fail. Task A: the median absolute percentage error is 14–16% in the middle price quartiles but 34% in the cheapest and 25% in the luxury quartile, where MAE is 259,600 AZN; by category it is 18% for flats (5,420 new, 1,801 old) but 41% for houses, 50% for commercial objects and 65% for land plots, whose price depends on unrecorded plot quality. Sparse districts (merged into "other", 4.7% of test rows) have median error 69% vs 21% elsewhere, and a tier error of 13.3% vs 8.7%. Task B: the tree errs on 25.2% of listings within ±15% of the threshold (19% of test rows) but

Table 3. Task B: premium vs standard (test set, mean ± bootstrap std; fit/predict time in s; CV = 5-fold F1).

Model	Acc.	Prec.	Rec.	F1	ROC-AUC	PR-AUC	fit / pred	CV F1
Ours: tree (entropy, d18)	.911 ± .003	.906	.913	.910 ± .003	.924 ± .003	.890	7.81 / 0.022	.915 ± .002
sklearn: tree	.911 ± .003	.906	.915	.910 ± .004	.923 ± .003	.889	0.32 / 0.004	.914 ± .002
Ours: SVM (Pegasos, linear)	.851 ± .004	.846	.854	.850 ± .004	.925 ± .003	.908	0.87 / 0.002	.846 ± .002
sklearn: SGD hinge [Pipeline]	.852 ± .004	.849	.849	.849 ± .004	.925 ± .003	.907	0.91 / 0.004	.844 ± .003
Ours: SVM (Pegasos + RFF, RBF)	.845 ± .004	.834	.855	.844 ± .004	.928 ± .003	.918	1.36 / 0.082	.846 ± .003
sklearn: SVC RBF [8k rows]	.852 ± .004	.842	.861	.851 ± .004	.933 ± .003	.923	1.82 / 2.41	.852 ± .002

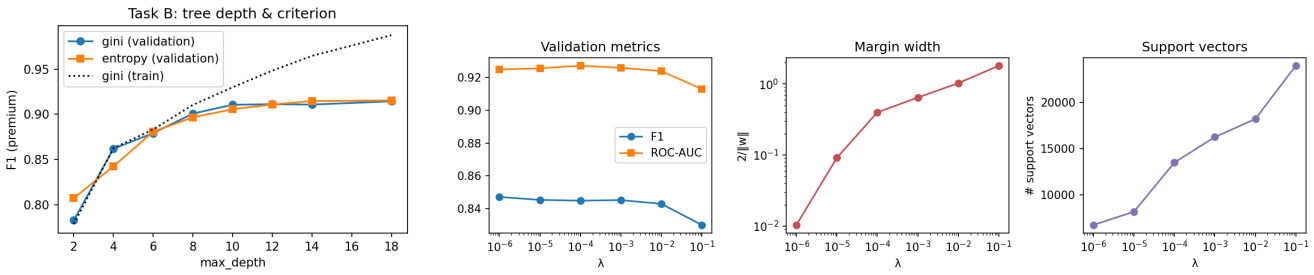


Figure 2. Left: Task B tree depth and criterion (validation F1). Right: SVM λ sweep: validation F1/AUC, margin width and number of support vectors.

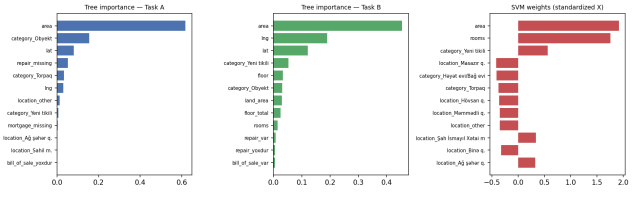


Figure 3. Tree importances (Task A, B) and SVM weights.

only 5.2% elsewhere, so most errors are unavoidable boundary cases.

Limitations. (i) $R^2 = 0.55$ on raw price is modest: RMSE is dominated by a few very expensive listings and a log target lowers MAE (100,915 vs 110,300) but not RMSE. (ii) Bootstrap std reflects test-set sampling, not training randomness; CV is run on train+val with hyper-parameters already chosen on validation, so it is not nested CV. (iii) `SVC` uses 8,000 training rows, so the RBF comparison favours neither side perfectly. (iv) Our tree is slower than scikit-learn. (v) Listing text and `views` were not used; asking prices are not transaction prices; the scrape is from 2024. (vi) The tolerance rule for λ was chosen after observing that the raw argmax ($\lambda = 10^{-6}$) differed from the others by only 0.002 F1 and destabilised the SGD baseline.

VIII. Conclusion

Our from-scratch tree and Pegasos SVM reproduce scikit-learn's accuracy on real, messy bina.az data (tree R^2 and F1 identical to three decimals; SVM within 0.002 F1 of `SGDClassifier`). The tree is the best classifier (F1 0.910 vs 0.850) because location is non-linear; the SVMs rank listings equally well (AUC $\approx$ 0.93). Next steps: ensembles of our trees, text features from the descriptions, and per-category models for land plots.

Tools and Acknowledgements

An AI assistant (Claude, Anthropic) was used to review the code against the project statement, to add the epochs/learning-rate-schedule options, near-duplicate removal and the validation-only selection rule, and to draft this report and the slides from `report/results.json`. All numbers come from `python -m src.run_all`; the team reviewed and can explain every change. Data: Kaggle *binaaz-sale-project* (sehriyarmemmedli). Baselines: scikit-learn.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems 20*, 2007.